

TEHNIČKI IZVEŠTAJ

Bash Script Check

Transformer za procenu bezbednosti shell skripti

Sistem analizira tekst skripte bez njenog izvršavanja i klasifikuje ga kao safe, risky ili malicious, uz razlog kada je rezultat rizičan ili zlonameran.

Skup podataka	Klase	Model	Vokabular
4.000	3 + 7 razloga	4 Transformer bloka	2.259 BPE tokena

Urediv Word dokument · 26.07.2026.

1. Sažetak projekta

Bash Script Check je eksperimentalni sistem za statičku procenu Bash i shell skripti. Ulaz se tretira kao običan tekst i nikada se ne izvršava. Sistem vraća jednu od tri glavne klase — safe, risky ili malicious — a za rizične i zlonamerne rezultate pokušava da navede i razumljiv razlog.

Najvažniji zaključak: Interni rezultat od 100% meri koliko dobro checkpoint prepoznaje izdvojene primere istog porekla. To nije isto što i 100% tačnost na skriptama iz realnog sveta. Sintetički obrasci, mali broj retkih kategorija i ograničen showcase skup zahtevaju oprezno tumačenje.

Ciljevi sistema

- Brzo izdvajanje očigledno bezbednih, rizičnih i zlonamernih obrazaca u tekstu skripte.
- Davanje razloga kao što su `reverse_shell`, `exfiltration`, `persistence`, `firewall_change` ili `infinite_loop`.
- Odvojeni CPU i CUDA GPU tokovi treninga, uz mogućnost samostalne predikcije iz sačuvanih artefakata.
- Prikaz neizvesnog rezultata umesto prinudne tvrdnje kada je maksimalna verovatnoća ispod praga 0,65.

Šta sistem nije

Ovo nije sandbox, antivirus niti formalni analizator toka izvršavanja. Model ne zna stvarne vrednosti promenljivih u runtime-u, ne prati mrežni saobraćaj i ne može da garantuje bezbednost. Oznaka safe je pomoćni signal za pregled, a ne dozvola za automatsko izvršavanje nepoznatog koda.

2. Šta je Transformer

Transformer je neuronska arhitektura koja obrađuje sekvencu tokena i uči veze između njenih delova pomoću mehanizma self-attention. Za ovaj projekat tokeni nisu reči prirodnog jezika, već delovi Bash komandi, putanja, operatora, zastavica i drugih znakova koda.

Self-attention ukratko

Svaki token formira vektore query, key i value. Sličnost query i key vektora određuje koliko pažnje jedan token daje drugom, dok se value vektori kombinuju u novu kontekstualnu reprezentaciju. Intuitivno, model može da poveže curl sa udaljenom adresom i osetljivom putanjom čak i kada između njih postoji više komandi.

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) \cdot V$$

Zašto su potrebne pozicije

Self-attention sam po sebi ne poznaje redosled. Sinusoidal positional encoding zato dodaje informaciju o poziciji svakom embedding vektoru. Redosled je kritičan u shell kodu: ista komanda pre ili posle pipe operatora može imati sasvim drugačije značenje.

Ostali delovi bloka

Komponenta	Uloga
Multi-head attention	Šest glava paralelno uči različite vrste veza između tokena.
Feed-forward mreža	Nelinearno transformiše reprezentaciju svakog položaja; unutrašnja dimenzija je 768.
Residual veze i LayerNorm	Stabilizuju protok gradijenta i omogućavaju treniranje dubljeg modela.
GELU i dropout	Dodaju nelinearnost i regularizaciju; dropout je 0,10.

3. Stvarna arhitektura u projektu

Tok obrade Bash skripte



Predikcija = neuronski model + pravila visokog poverenja + prag neizvesnosti

Rezultat: safe / risky / malicious / uncertain / invalid

Slika 1. Tok od ulaznog Bash teksta do glavne klase i objašnjenja.

Parametar	Vrednost	Značenje
Kontekst	384 tokena	Duži ulazi se skraćuju na zadatu granicu.
Dimenzija modela	192	Širina embedding i skrivenih vektora.
Attention glave	6	Paralelni pogledi na međuzavisnosti tokena.
Encoder blokovi	4	Dubina encoder-only Transformer mreže.
Feed-forward dimenzija	768	Kapacitet interne nelinearne transformacije.
Tokenizator	BPE, 2259 tokena	Byte-level BPE smanjuje problem nepoznatih komandi i adresa.
Pooling	masked mean	Prosek samo validnih, ne-padding tokena.
Izlazi	3 klase + 7 razloga	Dve linearne klasifikacione glave.

Dve izlazne glave

labelHead predviđa safe, risky ili malicious. reasonHead se trenira samo na zlonamernim uzorcima i predviđa jednu od sedam kategorija: defense_evasion, exfiltration, obfuscation, persistence, privilege_escalation, recon ili reverse_shell. Za risky klasu praktični razlog dolazi iz preciznih pravila ili opšte oznake model_detected_risk.

4. Zašto je izabran Transformer

Kriterijum	Prednost	Ograničenje
Kontekst	Povezuje udaljene delove skripte i lanac komandi.	Kontekst je ograničen na 384 tokena.
Redosled	Poziciono kodiranje čuva informaciju o sekvenci.	Ne modeluje potpuno semantiku izvršavanja.
Paralelizacija	Efikasno treniranje batch-eva na GPU-u.	Self-attention raste približno kvadratno sa dužinom ulaza.
Fleksibilnost	Jedan encoder podržava glavnu klasu i razlog.	Traži mnogo raznovrsnih i kvalitetno označenih podataka.
Rad sa kodom	BPE uči fragmente komandi, putanja i operatora.	Obfuskacija i dinamički generisan kod ostaju teški.

Zašto ne samo regex ili klasični model

Regex pravila su odlična za veoma karakteristične potpise, ali teško generalizuju na nove forme. Bag-of-words i slični klasični modeli gube veliki deo redosleda i konteksta. Projekat zato koristi hibrid: Transformer za statističku generalizaciju, a mali skup pravila samo za jasne, visokorizične obrasce i poznate administrativne izuzetke.

CPU i GPU tok

CPU tok je kompatibilan sa svakim podržanim računarom, ali je sporiji. GPU tok zahteva NVIDIA CUDA i ne prelazi tiho na CPU. Na CUDA uređaju se uključuju mixed precision, pinned memory i non-blocking transferi, što smanjuje vreme treninga i upotrebu memorije. Oba toka na kraju čuvaju iste tipove artefakata.

Artefakt	Svrha
artifacts/bash_transformer.pt	Težine modela, konfiguracija, redosled klasa, razlozi i ugrađeni tokenizer JSON.
artifacts/bash_tokenizer.json	Spoljašnja kopija tokenizatora radi pregleda i kompatibilnosti sa starijim checkpoint-ima.

5. Podaci: poreklo, raspodela i kvalitet

Raspodela glavnih klasa



Grafik 1. Raspodela 4.000 zapisa u podrazumevanom JSONL skupu.

Klasa	Broj	Udeo	Opis
safe	1500	37,5%	Rutinska automatizacija, provere, build, backup i administracija bez promene sistema.
risky	1500	37,5%	Privilegovane, destruktivne ili sistemske operacije koje mogu biti legitimne.
malicious	1000	25,0%	Komande povezane sa napadačkim ponašanjem i sedam kategorija razloga.

Šta repozitorijum zaista potvrđuje

generate_dataset.py lokalno proizvodi sintetičke safe i risky zapise iz parametrizovanih šablona. Identifikatori u kombinovanom skupu potvrđuju 1.500 safe-, 1.500 risky- i 1.000 RTO- zapisa. scrape_dataset.py postoji kao opcioni GitHub API kanal za realne install, deploy i admin skripte, ali trenutni kombinovani fajl nema identifikatore ni source URL polja koji dokazuju da su scrapeovani redovi ušli u ovaj konkretan checkpoint.

Napomena o Hugging Face poreklu: Korisnički opis navodi da RTO zlonamerni skup potiče sa Hugging Face-a. U trenutnom repozitorijumu ne postoji dataset URL, dataset card, autor ni licenca koji to mogu nezavisno da potvrde. Izveštaj zato tretira Hugging Face kao navedeno, ali nedokumentovano poreklo. Pre javne predaje treba dodati tačan link i uslove licence.

Zašto su sintetički i scrapeovani podaci korisni

Sintetički podaci brzo obezbeđuju balansirane klase, ponovljivost i kontrolu nad varijacijama. Scrapeovani kod donosi prirodniji stil, realne komentare, neočekivane kombinacije i šum. Najbolji skup kombinuje oba izvora, ali zahteva ručnu reviziju oznaka, proveru licenci, uklanjanje tajni i grupisanje po repozitorijumu ili porodici šablona pre podele.

6. Neravnoteža razloga

Raspodela razloga zlonamernog ponašanja



Crveno: kategorije sa premalo primera za pouzdanu procenu.

Grafik 2. Broj zlonamernih primera po kategoriji; dve kategorije su kritično male.

Glavne klase su relativno balansirane, ali razlozi nisu. recon ima 217 zapisa, dok obfuscation ima samo 4, a privilege_escalation samo 2. Zbog toga ukupna tačnost razloga može izgledati visoko iako model praktično nije pouzdano naučio dve retke kategorije.

Razlog	Broj	Procena
recon	217	upotrebljivo za eksperiment
exfiltration	195	upotrebljivo za eksperiment
reverse_shell	195	upotrebljivo za eksperiment
defense_evasion	194	upotrebljivo za eksperiment
persistence	193	upotrebljivo za eksperiment
obfuscation	4	kritično malo
privilege_escalation	2	kritično malo

Preporučena korekcija

- Prikupiti najmanje nekoliko stotina nezavisnih, ručno pregledanih primera za svaku kategoriju.
- Meriti precision, recall i F1 za svaki razlog, a ne samo ukupnu reason accuracy.
- Izvesti odvojeni test po izvoru: sintetički, GitHub i potpuno nezavisan ručno kuriran skup.

- Ne duplirati mali broj seed primera prostim menjanjem IP adrese ili putanje; to povećava broj redova, ali ne i semantičku raznovrsnost.

7. Trening i evaluacija

Parametar treninga	Vrednost	Zašto postoji
Epohe	12	Više prolaza kroz trening skup.
Batch size	32	Kompromis stabilnosti gradijenta i memorije.
Optimizer	AdamW	Adaptivni koraci uz odvojeni weight decay.
Learning rate	3×10^{-4}	Veličina ažuriranja parametara.
Weight decay	1×10^{-4}	Regularizacija težina.
Label smoothing	0,05	Smanjuje preteranu sigurnost modela.
Gradient clipping	1,0	Sprečava eksploziju gradijenta.
Reason loss weight	0,5	Balansira glavnu klasu i sekundarni razlog.
Split	20%, seed 42	Ponovljiv izdvojeni test.

Zaštita od curenja šablona

Nova podela izračunava template fingerprint: normalizuje URL-ove, IP adrese, brojeve i stringove, pa sve varijante iste porodice drži u istoj strani podele. U prethodnoj nasumičnoj podeli oko 26,8% test uzoraka imalo je porodicu šablona prisutnu u treningu; grupisana podela taj oblik curenja svodi na 0%. Ovo je jedna od najvažnijih korekcija projekta.

Optimizacije

- Tokenizacija se kešira u BashDataset umesto ponavljanja u svakoj epohi.
- Batch se dinamički dopunjava samo do najdužeg primera u tom batch-u.
- CUDA tok koristi mixed precision i GPU-friendly prenos memorije.
- Class weights ublažavaju neravnotežu glavnih klasa i razloga.

8. Rezultati i njihovo pravilno tumačenje

Matrica konfuzije — interni test

		Predviđena klasa		
		safe	risky	malicious
Stvarna klasa	safe	300	0	0
	risky	0	300	0
	malicious	0	0	201

Grafik 3. Sačuvana matrica konfuzije: redovi su stvarne, a kolone predviđene klase.

Metrika	Rezultat	Uzorci	Tumačenje
Accuracy	100,00%	801	Udeo tačnih glavnih klasifikacija.
Macro F1	100,00%	801	Jednak značaj za safe, risky i malicious.
Reason accuracy	98,51%	201	Tačnost razloga samo na malicious uzorcima.

Zašto 100% nije dokaz produkcijske spremnosti: Test je izdvojen i grupisan bolje nego ranije, ali i dalje deli generator, stil i poreklo sa trening podacima. Savršen skor je signal da je interni problem lak ili da postoje preostali tragovi izvora, a ne dokaz da model razume svaku Bash skriptu.

Metrike potrebne za realni svet

- Nezavisan, ručno označen test iz repozitorijuma koji nisu korišćeni ni za razvoj pravila ni za trening.
- Malicious recall, jer propušten zlonamerni primer može biti skuplji od lažnog alarma.
- Precision za safe klasu, odnosno koliko često je skripta zaista bezbedna kada sistem kaže safe.
- Coverage uz prag neizvesnosti i accuracy samo među pokrivenim primerima.

- Kalibracija verovatnoće (ECE), jer confidence treba da odgovara stvarnoj učestalosti greške.

9. Problemi pronađeni tokom razvoja i rešenja

Problem	Zašto je nastao	Uvedeno rešenje
94% testa, loše realne predikcije	Sintetički test je bio sličniji treningu od showcase skripti.	Nezavisni showcase manifest, grupisani split i jasna ograničenja metrike.
admin_bash označen kao persistence	Model je mešao administrativne komande sa obrascima iz malicious skupa.	Usko read-only DNS pravilo ga označava safe samo bez mutirajućih komandi.
Reverse shell označen safe	Statistički model nije naučio sve eksplicitne potpise.	Pravila visokog poverenja za /dev/tcp i netcat shell.
safe.txt je bio CSV	Ceo tabelarni fajl posmatran je kao jedna skripta.	CSV kontejner se prepoznaje i svaki script_content red se klasifikuje odvojeno.
Beskonačna petlja	Nije postojala posebna kategorija za busy loop.	Unbounded loop bez sleep/exit/break dobija risky / infinite_loop.
Rizičan rezultat bez razloga	Reason head pokriva samo malicious kategorije.	Risky pravila objašnjavaju firewall, brisanje, privilegije, servise i dozvole.
Nepoznati tokeni	Word-level pristup loše radi sa novim putanjama, IP adresama i zastavicama.	Byte-level BPE uči manje fragmente i može kodirati svaki bajt.
Nizak confidence predstavljen kao činjenica	Argmax uvek vraća klasu.	Prag 0,65 vraća uncertain i prikazuje top model label.
Checkpoint/tokenizer neusaglašenost	Odvojeni fajlovi mogu pripadati različitim treninzima.	Tokenizer JSON je ugrađen u novi checkpoint; prediction koristi artifacts putanju.

10. Predikcija i interpretacija rezultata

Komanda `python main.py predict` učitava `artifacts/bash_transformer.pt` jednom, prolazi kroz podržane fajlove u `showcase_scripts` i prikazuje ime, oznaku, confidence i razlog. Za CSV kontejner prikazuje i ime svakog ugrađenog zapisa.

Izlaz	Značenje	Preporučena reakcija
safe	Nisu pronađeni poznati opasni obrasci; model smatra ulaz niskorizičnim.	I dalje pregledati nepoznat kod pre izvršavanja.
risky	Legitimna administracija sa potencijalno destruktivnim ili privilegovanim efektom.	Tražiti odobrenje i proveriti reason/explanation.
malicious	Napadački obrazac ili jasna sigurnosna signatura.	Ne izvršavati; izolovati i analizirati.
uncertain	Najbolja verovatnoća modela je ispod 0,65.	Obavezna ručna procena; ne pretvarati u safe.
invalid	Prazan ili nepodržan ulaz.	Ispraviti format ili izdvojiti pojedinačne skripte.

Confidence nije verovatnoća bezbednosti

Confidence je softmax vrednost za izabranu klasu u odnosu na druge dve klase. Visoka vrednost može biti pogrešna kada je ulaz van distribucije treninga. Pravila vraćaju 0,99 ili 0,999 kao projektovanu oznaku visokog poverenja, ali ni te vrednosti nisu empirijska garancija.

Primer: persistence

Persistence znači pokušaj da se pristup ili izvršavanje zadrži kroz nove sesije ili restart, na primer preko `cron-a`, `systemd servisa`, `.bashrc fajla` ili `authorized_keys`. Običan `read-only admin skript` nije persistence samo zato što koristi sistemske komande. Zato je DNS izuzetak namerno uzak i proverava da nema `sudo`, `curl`, `wget`, `firewall`, `service`, `crontab`, `eval` i drugih mutirajućih ili opasnih komandi.

11. Ograničenja i preporučeni sledeći koraci

Prioritet	Akcija	Očekivani efekat
1	Napraviti nezavisan ručno označen test od najmanje nekoliko stotina skripti iz novih repozitorijuma.	Realnija procena precision/recall i manje oslanjanje na sintetički skor.
2	Dodati tačan Hugging Face dataset URL, dataset card, autora i licencu; čuvati source polje po zapisu.	Reproduktivno i pravno jasnije poreklo podataka.
3	Dopuniti obfuscation i privilege_escalation raznovrsnim primerima.	Smislenija tačnost razloga i manje pristrasnosti prema čestim klasama.
4	Podeliti realne podatke po repozitorijumu/organizaciji, ne po pojedinačnom fajlu.	Sprečavanje source leakage-a.
5	Dodati adversarial testove: kodiranje, eval, heredoc, alias, razlomljene komande i duge skripte.	Otpornost na obfuskaciju i zaobilaženje pravila.
6	Kalibrisati threshold na validacionom skupu i prikazivati sve tri verovatnoće u analitičkom režimu.	Bolje upravljanje uncertain rezultatima.
7	Voditi model card sa datumom, hash-om podataka, konfiguracijom i metrikama svakog checkpoint-a.	Praćenje verzija i lakša reprodukcija.

Zaključak

Projekat sada ima jasnu encoder-only Transformer arhitekturu, odvojene CPU/GPU tokove, artefakte vezane za projekat, samostalnu predikciju i praktične zaštite za očigledne bezbednosne slučajeve i napade. Najveći dalji dobitak neće doći samo od većeg modela, već od kvalitetnijeg, nezavisnog i dokumentovanog skupa podataka. U sadašnjem obliku sistem je dobar eksperimentalni pomoćnik za trijažu i demonstraciju mašinskog učenja nad kodom, uz obaveznu ljudsku proveru.

Dodatak A — Mapa modula

Fajl	Odgovornost
main.py	Tri javna workflow poziva: CPU, GPU i prediction.
bash_classifier/config.py	Klase, konfiguracije modela i treninga, tekstualna objašnjenja razloga.
bash_classifier/data.py	JSONL validacija, grupisani split, BPE, Dataset i DataLoader.
bash_classifier/model.py	Transformer, dve glave, save/load checkpoint logika.
bash_classifier/training.py	Loss funkcije, optimizer tok, epohe i čuvanje modela.
bash_classifier/cpu_training.py	Eksplisitni CPU ulaz u trening.
bash_classifier/gpu_training.py	CUDA proveru i mixed-precision GPU ulaz.
bash_classifier/evaluation.py	Matrice konfuzije, F1, kalibracija i showcase evaluacija.
bash_classifier/prediction.py	Predikcija, prag confidence-a, CSV ekstrakcija i format izlaza.
bash_classifier/security_rules.py	Uska pravila za opasne, rizične i poznate safe obrasce.
generate_dataset.py	Sintetičko generisanje safe/risky podataka i kombinovanje JSONL fajlova.
scrape_dataset.py	Opcioni GitHub API scraping realnih administrativnih skripti.

Pravila imenovanja

Kebab-case nije validan za Python identifikatore jer znak - znači oduzimanje. Zbog toga funkcije koriste snake_case (na primer train_model), promenljive i parametri camelCase (trainingConfig), a konstante UPPER_SNAKE_CASE. Svaka aplikaciona funkcija treba da ima docstring odmah ispod definicije koji objašnjava šta radi i zašto postoji.

Dodatak B — Funkcije i njihova uloga

Sledeće tabele obuhvataju svih 117 funkcija pronađenih u projektnim Python modulima. Linija označava mesto definicije u trenutnoj lokalnoj verziji koda.

bash_classifier/cli.py

Funkcija	Linija	Šta radi / zašto postoji
build_argument_parser()	13	Kreira i konfiguriše „argument parser“ kao deo toka u modulu bash_classifier/cli.py.

bash_classifier/config.py

Funkcija	Linija	Šta radi / zašto postoji
model_config_from_dict()	48	Pretvara rečnik iz checkpoint-a u ModelConfig i podržava starija snake_case imena.
training_config_from_dict()	62	Rekonstruiše TrainingConfig iz checkpoint rečnika i normalizuje starija imena polja.

bash_classifier/cpu_training.py

Funkcija	Linija	Šta radi / zašto postoji
train_on_cpu()	8	Trenira ili koordinira trening za „on cpu“ kao deo toka u modulu bash_classifier/cpu_training.py.

bash_classifier/data.py

Funkcija	Linija	Šta radi / zašto postoji
load_jsonl()	22	Učitava i proverava „jsonl“ kao deo toka u modulu bash_classifier/data.py.
stratified_split()	45	Pomoćna funkcija „stratified split“ implementira internu operaciju potrebnu modulu bash_classifier/data.py.
template_fingerprint()	80	Normalizuje promenljive detalje skripte da bi varijante istog šablona ostale u istoj strani podele.
get_all_scripts()	92	Vraća „all scripts“ kao deo toka u modulu bash_classifier/data.py.
build_tokenizer()	98	Kreira i konfiguriše „tokenizer“ kao deo toka u modulu bash_classifier/data.py.
get_reason_names()	121	Vraća „reason names“ kao deo toka u modulu bash_classifier/data.py.
required_token_id()	133	Pomoćna funkcija „required token id“ implementira internu operaciju potrebnu modulu bash_classifier/data.py.
__init__()	144	Pomoćna funkcija „init“ implementira internu operaciju potrebnu modulu bash_classifier/data.py.
__len__()	160	Pomoćna funkcija „len“ implementira internu operaciju potrebnu modulu bash_classifier/data.py.

Funkcija	Linija	Šta radi / zašto postoji
<code>__getitem__()</code>	164	Pomoćna funkcija „getitem“ implementira internu operaciju potrebnu modulu <code>bash_classifier/data.py</code> .
<code>collate_batch()</code>	177	Dinamički dopunjava sekvence samo do najdužeg primera u batch-u i pravi attention masku.
<code>make_data_loader()</code>	194	Kreira „data loader“ kao deo toka u modulu <code>bash_classifier/data.py</code> .
<code>move_batch_to_device()</code>	216	Pomoćna funkcija „move batch to device“ implementira internu operaciju potrebnu modulu <code>bash_classifier/data.py</code> .
<code>prepare_split_data()</code>	226	Priprema „split data“ kao deo toka u modulu <code>bash_classifier/data.py</code> .
<code>prepare_evaluation_data()</code>	258	Priprema „evaluation data“ kao deo toka u modulu <code>bash_classifier/data.py</code> .

bash_classifier/devices.py

Funkcija	Linija	Šta radi / zašto postoji
<code>choose_inference_device()</code>	8	Bira „inference device“ kao deo toka u modulu <code>bash_classifier/devices.py</code> .

bash_classifier/evaluation.py

Funkcija	Linija	Šta radi / zašto postoji
<code>calculate_metrics()</code>	18	Iz matrice konfuzije računa accuracy, balanced accuracy, macro F1, kalibraciju i reason accuracy.
<code>test_model()</code>	71	Testira „model“ kao deo toka u modulu <code>bash_classifier/evaluation.py</code> .
<code>print_metrics()</code>	126	Formatira i ispisuje „metrics“ kao deo toka u modulu <code>bash_classifier/evaluation.py</code> .
<code>test_saved_model()</code>	150	Testira „saved model“ kao deo toka u modulu <code>bash_classifier/evaluation.py</code> .
<code>evaluate_saved_model()</code>	175	Evaluiira „saved model“ kao deo toka u modulu <code>bash_classifier/evaluation.py</code> .
<code>evaluate_real_world_examples()</code>	190	Poredi kompletan prediction sistem sa ručno zadatim očekivanjima iz showcase manifesta.

bash_classifier/gpu_training.py

Funkcija	Linija	Šta radi / zašto postoji
<code>require_cuda_device()</code>	8	Pomoćna funkcija „require cuda device“ implementira internu operaciju potrebnu modulu <code>bash_classifier/gpu_training.py</code> .
<code>train_on_gpu()</code>	21	Trenira ili koordinira trening za „on gpu“ kao deo toka u modulu <code>bash_classifier/gpu_training.py</code> .

bash_classifier/model.py

Funkcija	Linija	Šta radi / zašto postoji
<code>__init__()</code>	25	Pomoćna funkcija „init“ implementira internu operaciju potrebnu modulu <code>bash_classifier/model.py</code> .

Funkcija	Linija	Šta radi / zašto postoji
forward()	38	Izvršava forward prolaz odgovarajućeg PyTorch modula i vraća njegove izlazne tenzore.
__init__()	46	Pomoćna funkcija „init“ implementira internu operaciju potrebnu modulu bash_classifier/model.py.
initialize_parameters()	84	Inicijalizuje matrične težine Xavier uniform postupkom radi stabilnog početka treniranja.
forward()	90	Izvršava forward prolaz odgovarajućeg PyTorch modula i vraća njegove izlazne tenzore.
make_model()	102	Kreira „model“ kao deo toka u modulu bash_classifier/model.py.
save_checkpoint()	117	Čuva „checkpoint“ kao deo toka u modulu bash_classifier/model.py.
normalize_state_dict()	140	Normalizuje „state dict“ kao deo toka u modulu bash_classifier/model.py.
load_checkpoint()	157	Učitava i proverava „checkpoint“ kao deo toka u modulu bash_classifier/model.py.

bash_classifier/prediction.py

Funkcija	Linija	Šta radi / zašto postoji
predict_text()	24	Kombinuje validaciju, pravila visokog poverenja i Transformer predikciju sa pragom neizvesnosti.
predict_showcase_directory()	100	Predviđa „showcase directory“ kao deo toka u modulu bash_classifier/prediction.py.
extract_tabular_scripts()	166	Izdvađa „tabular scripts“ kao deo toka u modulu bash_classifier/prediction.py.
format_prediction_summary()	188	Formatira „prediction summary“ kao deo toka u modulu bash_classifier/prediction.py.

bash_classifier/security_rules.py

Funkcija	Linija	Šta radi / zašto postoji
validate_script_text()	103	Proverava ispravnost „script text“ kao deo toka u modulu bash_classifier/security_rules.py.
find_security_rule()	113	Traži eksplicitne malicious/risky potpise i uske safe izuzetke pre statističkog modela.
contains_unbounded_busy_loop()	155	Proverava da li ulaz sadrži „unbounded busy loop“ kao deo toka u modulu bash_classifier/security_rules.py.
is_read_only_dns_diagnostic()	170	Proverava uslov za „read only dns diagnostic“ kao deo toka u modulu bash_classifier/security_rules.py.
explain_risky_behavior()	180	Vraća najkonkretniji poznati razlog za risky rezultat ili neutralni model_detected_risk fallback.

bash_classifier/training.py

Funkcija	Linija	Šta radi / zašto postoji
----------	--------	--------------------------

Funkcija	Linija	Šta radi / zašto postoji
set_seed()	18	Postavlja „seed“ kao deo toka u modulu bash_classifier/training.py.
class_weights()	26	Pomoćna funkcija „class weights“ implementira internu operaciju potrebnu modulu bash_classifier/training.py.
reason_class_weights()	34	Pomoćna funkcija „reason class weights“ implementira internu operaciju potrebnu modulu bash_classifier/training.py.
make_gradient_scaler()	57	Kreira „gradient scaler“ kao deo toka u modulu bash_classifier/training.py.
train_epoch()	64	Trenira ili koordinira trening za „epoch“ kao deo toka u modulu bash_classifier/training.py.
train_model()	112	Trenira ili koordinira trening za „model“ kao deo toka u modulu bash_classifier/training.py.
train_new_model()	142	Trenira ili koordinira trening za „new model“ kao deo toka u modulu bash_classifier/training.py.

generate_dataset.py

Funkcija	Linija	Šta radi / zašto postoji
pick()	51	Deterministički bira jednu vrednost iz ponuđenih varijanti sintetičkog generatora.
header()	55	Pomoćna funkcija „header“ implementira internu operaciju potrebnu modulu generate_dataset.py.
safe_greet()	67	Generiše sintetičku safe skriptu za scenario „greet“ radi raznovrsnijeg trening skupa.
safe_list_dir()	76	Generiše sintetičku safe skriptu za scenario „list dir“ radi raznovrsnijeg trening skupa.
safe_git_clone()	90	Generiše sintetičku safe skriptu za scenario „git clone“ radi raznovrsnijeg trening skupa.
safe_py_venv()	105	Generiše sintetičku safe skriptu za scenario „py venv“ radi raznovrsnijeg trening skupa.
safe_npm_build()	118	Generiše sintetičku safe skriptu za scenario „npm build“ radi raznovrsnijeg trening skupa.
safe_make_build()	129	Generiše sintetičku safe skriptu za scenario „make build“ radi raznovrsnijeg trening skupa.
safe_backup_home()	139	Generiše sintetičku safe skriptu za scenario „backup home“ radi raznovrsnijeg trening skupa.
safe_log_scan()	152	Generiše sintetičku safe skriptu za scenario „log scan“ radi raznovrsnijeg trening skupa.
safe_env_report()	167	Generiše sintetičku safe skriptu za scenario „env report“ radi raznovrsnijeg trening skupa.
safe_health_check()	179	Generiše sintetičku safe skriptu za scenario „health check“ radi raznovrsnijeg trening skupa.
safe_csv_summary()	194	Generiše sintetičku safe skriptu za scenario „csv summary“ radi raznovrsnijeg trening skupa.
safe_rename_batch()	205	Generiše sintetičku safe skriptu za scenario „rename batch“ radi raznovrsnijeg trening skupa.
safe_wait_for_port()	218	Generiše sintetičku safe skriptu za scenario „wait for port“ radi raznovrsnijeg trening skupa.

Funkcija	Linija	Šta radi / zašto postoji
safe_json_pretty()	236	Generiše sintetičku safe skriptu za scenario „json pretty“ radi raznovrsnijeg trening skupa.
safe_disk_usage()	252	Generiše sintetičku safe skriptu za scenario „disk usage“ radi raznovrsnijeg trening skupa.
safe_find_large()	259	Generiše sintetičku safe skriptu za scenario „find large“ radi raznovrsnijeg trening skupa.
safe_checksum()	266	Generiše sintetičku safe skriptu za scenario „checksum“ radi raznovrsnijeg trening skupa.
safe_download_only()	273	Generiše sintetičku safe skriptu za scenario „download only“ radi raznovrsnijeg trening skupa.
safe_ping_check()	282	Generiše sintetičku safe skriptu za scenario „ping check“ radi raznovrsnijeg trening skupa.
safe_tar_extract()	293	Generiše sintetičku safe skriptu za scenario „tar extract“ radi raznovrsnijeg trening skupa.
safe_grep_todo()	302	Generiše sintetičku safe skriptu za scenario „grep todo“ radi raznovrsnijeg trening skupa.
safe_count_files()	309	Generiše sintetičku safe skriptu za scenario „count files“ radi raznovrsnijeg trening skupa.
safe_uptime_log()	317	Generiše sintetičku safe skriptu za scenario „uptime log“ radi raznovrsnijeg trening skupa.
safe_git_status()	324	Generiše sintetičku safe skriptu za scenario „git status“ radi raznovrsnijeg trening skupa.
safe_lint()	335	Generiše sintetičku safe skriptu za scenario „lint“ radi raznovrsnijeg trening skupa.
safe_gen_password()	342	Generiše sintetičku safe skriptu za scenario „gen password“ radi raznovrsnijeg trening skupa.
safe_watch_file()	348	Generiše sintetičku safe skriptu za scenario „watch file“ radi raznovrsnijeg trening skupa.
safe_json_field()	354	Generiše sintetičku safe skriptu za scenario „json field“ radi raznovrsnijeg trening skupa.
risky_apt_install()	376	Generiše sintetičku risky skriptu za scenario „apt install“ radi učenja potencijalno opasne administracije.
risky_curl_bash()	387	Generiše sintetičku risky skriptu za scenario „curl bash“ radi učenja potencijalno opasne administracije.
risky_add_apt_repo()	395	Generiše sintetičku risky skriptu za scenario „add apt repo“ radi učenja potencijalno opasne administracije.
risky_systemd_unit()	406	Generiše sintetičku risky skriptu za scenario „systemd unit“ radi učenja potencijalno opasne administracije.
risky_rm_cleanup()	429	Generiše sintetičku risky skriptu za scenario „rm cleanup“ radi učenja potencijalno opasne administracije.
risky_chmod_777()	440	Generiše sintetičku risky skriptu za scenario „chmod 777“ radi učenja potencijalno opasne administracije.
risky_dd_image()	450	Generiše sintetičku risky skriptu za scenario „dd image“ radi učenja potencijalno opasne administracije.
risky_mkfs()	461	Generiše sintetičku risky skriptu za scenario „mkfs“ radi učenja potencijalno opasne administracije.

Funkcija	Linija	Šta radi / zašto postoji
risky_iptables_flush()	473	Generiše sintetičku risky skriptu za scenario „iptables flush“ radi učenja potencijalno opasne administracije.
risky_ufw_disable()	485	Generiše sintetičku risky skriptu za scenario „ufw disable“ radi učenja potencijalno opasne administracije.
risky_add_user()	495	Generiše sintetičku risky skriptu za scenario „add user“ radi učenja potencijalno opasne administracije.
risky_docker_prune()	507	Generiše sintetičku risky skriptu za scenario „docker prune“ radi učenja potencijalno opasne administracije.
risky_docker_privileged()	516	Generiše sintetičku risky skriptu za scenario „docker privileged“ radi učenja potencijalno opasne administracije.
risky_crontab_install()	529	Generiše sintetičku risky skriptu za scenario „crontab install“ radi učenja potencijalno opasne administracije.
risky_swap_setup()	539	Generiše sintetičku risky skriptu za scenario „swap setup“ radi učenja potencijalno opasne administracije.
risky_sysctl_tune()	551	Generiše sintetičku risky skriptu za scenario „sysctl tune“ radi učenja potencijalno opasne administracije.
risky_deploy_release()	564	Generiše sintetičku risky skriptu za scenario „deploy release“ radi učenja potencijalno opasne administracije.
risky_disable_selinux()	578	Generiše sintetičku risky skriptu za scenario „disable selinux“ radi učenja potencijalno opasne administracije.
strip_comments()	596	Pomoćna funkcija „strip comments“ implementira internu operaciju potrebnu modulu generate_dataset.py.
generate()	608	Generiše traženi broj sintetičkih zapisa iz raspoloživih safe ili risky šablona.
write_jsonl()	633	Upisuje zapise u JSON Lines format, po jedan JSON objekat u svakom redu.
read_jsonl()	640	Čita JSON Lines zapise u memoriju radi normalizacije ili kombinovanja skupa.
read_malicious_jsonl()	645	Pomoćna funkcija „read malicious jsonl“ implementira internu operaciju potrebnu modulu generate_dataset.py.
main()	666	Predstavlja komandni ulaz skripte: parsira opcije i pokreće odgovarajući tok.

main.py

Funkcija	Linija	Šta radi / zašto postoji
train_test_evaluate_cpu()	11	Pokreće kompletan CPU tok: trening, test izdvojene podele i showcase evaluaciju.
train_test_evaluate_gpu()	20	Pokreće isti kompletan tok na CUDA GPU-u i namerno prekida ako CUDA nije dostupna.
predict_showcase_scripts()	29	Učitava sačuvani model iz artifacts direktorijuma i prikazuje predikciju za svaki showcase ulaz.

scrape_dataset.py

Funkcija	Linija	Šta radi / zašto postoji
strip_comments()	89	Pomoćna funkcija „strip comments“ implementira internu operaciju potrebnu modulu scrape_dataset.py.

Funkcija	Linija	Šta radi / zašto postoji
classify()	100	Pomoćna funkcija „classify“ implementira internu operaciju potrebnu modulu scrape_dataset.py.
gh_request()	109	Poziva GitHub API sa kontrolom grešaka, autentikacije i ograničenja zahteva.
search_code()	119	Pomoćna funkcija „search code“ implementira internu operaciju potrebnu modulu scrape_dataset.py.
fetch_content()	142	Pomoćna funkcija „fetch content“ implementira internu operaciju potrebnu modulu scrape_dataset.py.
main()	157	Predstavlja komandni ulaz skripte: parsira opcije i pokreće odgovarajući tok.
label_full()	171	Heuristički procenjuje oznaku kompletnog scrapeovanog shell teksta.

Dodatak C — Izvori unutar projekta

Ovaj izveštaj je generisan iz lokalne verzije repozitorijuma. Nisu korišćene niti izmišljene spoljne reference koje nisu dokumentovane u projektu.

Izvor	Korišćena informacija
README.md	Namena, upozorenja, tokovi pokretanja, tumačenje accuracy-ja i artefakti.
bash_classifier/*.py	Arhitektura, BPE, trening, evaluacija, predikcija, sigurnosna pravila i docstring komentari.
data/safe_risky_combined.jsonl	Broj zapisa, raspodela klasa, RTO identifikatori i kategorije razloga.
artifacts/bash_tokenizer.json	Tip tokenizatora i veličina rečnika.
tmp/pdfs/metrics.json	Sačuvana matrica konfuzije i interni rezultati checkpoint-a.
generate_dataset.py	Potvrda sintetičkog safe/risky generatora.
scrape_dataset.py	Potvrda opcionog GitHub scraping toka i njegovih ograničenja.

Provenance zadatak pre finalne predaje: Upisati tačan naziv i URL Hugging Face skupa, autora/verziju, licencu i datum preuzimanja. Ako je neki GitHub scrape stvarno korišćen, sačuvati source_url i license po svakom zapisu.